

–LOGGING FEATURE

```
CREATE SEQUENCE IF NOT EXISTS bl_cl.seq_mta_load_logs_id START WITH 1 INCREMENT BY 1;

-- log table creation

CREATE TABLE IF NOT EXISTS bl_cl.mta_load_logs (
  log_id          BIGINT DEFAULT nextval('bl_cl.seq_mta_load_logs_id') NOT NULL,
  log_dt          TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
  procedure_name  VARCHAR NOT NULL,
  log_status      VARCHAR NOT NULL,
  rows_affected   INT,
  log_message     TEXT,
  CONSTRAINT pk_mta_load_logs PRIMARY KEY (log_id),
  CONSTRAINT chk_mta_load_logs_status CHECK (log_status IN ('SUCCESS','ERROR'))
);
-- procedure to log outcome of tables insert/update data

CREATE OR REPLACE PROCEDURE bl_cl.p_log(
  p_procedure_name VARCHAR,
  p_log_status      VARCHAR,
  p_rows_affected   INT,
  p_log_message     TEXT
)
LANGUAGE plpgsql
AS $$
BEGIN
  INSERT INTO bl_cl.mta_load_logs (procedure_name, log_status, rows_affected, log_message)
  VALUES (p_procedure_name, p_log_status, p_rows_affected, p_log_message);
END;
$$;

-- return table type function to display log table
drop function bl_cl.get_load_summary
CREATE OR REPLACE FUNCTION bl_cl.get_load_summary()
RETURNS TABLE (
  log_id          BIGINT,
  log_dt          TIMESTAMP,
  procedure_name  VARCHAR,
  log_status      VARCHAR,
  rows_affected   INT,
  log_message     TEXT
)
LANGUAGE plpgsql
AS $$
BEGIN
  RETURN QUERY
  SELECT l.log_id, l.log_dt, l.procedure_name, l.log_status, l.rows_affected, l.log_message
  FROM bl_cl.mta_load_logs l
  ORDER BY l.log_id DESC;
END;
$$;

SELECT * FROM bl_cl.get_load_summary();
```

123 log_ic ▼	🕒 log_dt ▼	AZ procedure_nar ▼	AZ I ▼	123 rows_affected ▼	AZ log_message ▼
49	2026-07-19 13:04:46.803	load_ce_cities	SUCCESS	0	Inserted: 0, Updated: 0
48	2026-07-19 13:04:45.555	load_ce_cities	SUCCESS	194	Inserted: 194, Updated: 0
47	2026-07-19 13:03:41.755	load_ce_states	SUCCESS	0	Inserted: 0, Updated: 0
46	2026-07-19 13:03:40.607	load_ce_states	SUCCESS	59	Inserted: 59, Updated: 0
45	2026-07-19 12:26:43.660	load_ce_states	SUCCESS	9	Inserted: 9, Updated: 0
44	2026-07-19 12:22:14.669	load_ce_cities	SUCCESS	0	Inserted: 0, Updated: 0
43	2026-07-19 12:22:02.414	load_ce_cities	SUCCESS	0	Inserted: 0, Updated: 0
42	2026-07-19 12:22:00.407	load_ce_cities	SUCCESS	0	Inserted: 0, Updated: 0
41	2026-07-19 12:21:46.360	load_ce_states	SUCCESS	9	Inserted: 9, Updated: 0
40	2026-07-19 12:16:45.815	load_ce_cities	SUCCESS	0	Inserted: 0, Updated: 0
39	2026-07-19 12:16:44.296	load_ce_cities	SUCCESS	150	Inserted: 150, Updated: 0
38	2026-07-19 12:09:10.039	load_ce_cities	SUCCESS	0	Inserted: 0, Updated: 0
37	2026-07-19 12:09:03.846	load_ce_cities	SUCCESS	150	Inserted: 150, Updated: 0
36	2026-07-19 08:46:07.078	load_ce_cities	SUCCESS	8	Inserted: 0, Updated: 8
35	2026-07-19 08:46:05.740	load_ce_cities	SUCCESS	194	Inserted: 194, Updated: 0
34	2026-07-19 08:45:15.336	load_ce_cities	ERROR	[NULL]	relation "bl_3nf.ce_cities" c
33	2026-07-19 08:45:14.555	load_ce_cities	ERROR	[NULL]	relation "bl_3nf.ce_cities" c
32	2026-07-19 08:41:39.606	load_ce_cities	SUCCESS	8	Inserted: 0, Updated: 8
31	2026-07-19 08:41:37.738	load_ce_cities	SUCCESS	150	Inserted: 0, Updated: 150
30	2026-07-19 08:33:59.678	load_ce_states	SUCCESS	0	Inserted: 0, Updated: 0
29	2026-07-19 08:33:59.149	load_ce_states	SUCCESS	59	Inserted: 59, Updated: 0
28	2026-07-19 08:32:27.206	load_ce_states	ERROR	[NULL]	relation "bl_3nf.ce_states"
27	2026-07-19 08:32:27.015	load_ce_states	ERROR	[NULL]	relation "bl_3nf.ce_states"
26	2026-07-19 08:23:53.123	load_ce_states	SUCCESS	0	Inserted: 0, Updated: 0
25	2026-07-19 08:23:52.443	load_ce_states	SUCCESS	0	Inserted: 0, Updated: 0
24	2026-07-19 08:21:04.956	load_ce_states	ERROR	[NULL]	column "_state" does not e
23	2026-07-19 08:21:04.431	load_ce_states	ERROR	[NULL]	column "_state" does not e
22	2026-07-19 08:18:40.026	load_ce_states	SUCCESS	0	Inserted: 0, Updated: 0
21	2026-07-19 08:18:33.113	load_ce_states	SUCCESS	9	Inserted: 9, Updated: 0
20	2026-07-19 07:58:04.417	load_ce_cities	SUCCESS	8	Inserted: 0, Updated: 8
19	2026-07-19 07:58:03.570	load_ce_cities	SUCCESS	194	Inserted: 194, Updated: 0
18	2026-07-19 07:57:39.729	load_ce_products	SUCCESS	0	Inserted: 0, Updated: 0
17	2026-07-19 07:57:38.755	load_ce_products	SUCCESS	5,000	Inserted: 5000, Updated: 0
16	2026-07-19 07:57:13.138	load_ce_subchannels	SUCCESS	0	Inserted: 0, Updated: 0
15	2026-07-19 07:57:11.929	load_ce_subchannels	SUCCESS	12	Inserted: 12, Updated: 0
14	2026-07-19 07:55:31.169	load_ce_states	SUCCESS	0	Inserted: 0, Updated: 0
13	2026-07-19 07:55:30.276	load_ce_states	SUCCESS	50	Inserted: 50, Updated: 0
12	2026-07-19 07:54:58.673	load_ce_subcategories	SUCCESS	0	Inserted: 0, Updated: 0
11	2026-07-19 07:54:57.620	load_ce_subcategories	SUCCESS	64	Inserted: 64, Updated: 0
10	2026-07-19 07:54:09.482	load_ce_customers_scc	SUCCESS	0	Inserted: 0, Expired: 0
9	2026-07-19 07:53:58.064	load_ce_customers_scc	SUCCESS	289,206	Inserted: 289206, Expired:
8	2026-07-19 07:53:21.192	load_ce_employees	SUCCESS	0	Inserted: 0, Updated: 0
7	2026-07-19 07:53:20.348	load_ce_employees	SUCCESS	200	Inserted: 200, Updated: 0
6	2026-07-19 07:52:49.241	load_ce_channels	SUCCESS	0	Inserted: 0, Updated: 0 (FC
5	2026-07-19 07:52:48.396	load_ce_channels	SUCCESS	6	Inserted: 6, Updated: 0 (FC
4	2026-07-19 07:52:15.681	load_ce_countries	SUCCESS	0	Inserted: 0, Updated: 0
3	2026-07-19 07:52:15.007	load_ce_countries	SUCCESS	10	Inserted: 10, Updated: 0
2	2026-07-19 07:51:19.665	load_ce_categories	SUCCESS	0	Inserted: 0, Updated: 0
1	2026-07-19 07:51:19.271	load_ce_categories	SUCCESS	16	Inserted: 16, Updated: 0

-CE_CATEGORIES

Standard two-source load (INT + US). Idempotent via NOT EXISTS (insert) and IS DISTINCT FROM (update).

```
-- CE_CATEGORIES
CREATE OR REPLACE PROCEDURE bl_cl.load_ce_categories()
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows_ins INT := 0;
    v_rows_upd INT := 0;
    v_tmp      INT := 0;
BEGIN
    -- INT
    UPDATE bl_3nf.ce_categories tgt
    SET product_category_name = src.category_name, update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(category, ''), 'n. a.') AS category_name,
            COALESCE(NULLIF(category, ''), 'n. a.') AS category_src_id
          FROM sa_int_sales.src_int_sales WHERE category IS NOT NULL) src
    WHERE tgt.product_category_name_src_id = src.category_src_id
      AND tgt.source_system = 'SA_INT_SALES'
      AND tgt.product_category_name IS DISTINCT FROM src.category_name;
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;

    INSERT INTO bl_3nf.ce_categories (product_category_id, product_category_name,
    source_system, source_entity, product_category_name_src_id, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_categories_id'), src.category_name,
    'SA_INT_SALES', 'SRC_INT_SALES', src.category_src_id, CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(category, ''), 'n. a.') AS category_name,
            COALESCE(NULLIF(category, ''), 'n. a.') AS category_src_id
          FROM sa_int_sales.src_int_sales WHERE category IS NOT NULL) src
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_categories tgt
    WHERE tgt.product_category_name_src_id = src.category_src_id
      AND tgt.source_system = 'SA_INT_SALES');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    -- US
    UPDATE bl_3nf.ce_categories tgt
    SET product_category_name = src.category_name, update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(product_category, ''), 'n. a.') AS category_name,
            COALESCE(NULLIF(product_category, ''), 'n. a.') AS category_src_id
          FROM sa_us_sales.src_us_sales WHERE product_category IS NOT NULL) src
    WHERE tgt.product_category_name_src_id = src.category_src_id
      AND tgt.source_system = 'SA_US_SALES'
      AND tgt.product_category_name IS DISTINCT FROM src.category_name;
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;

    INSERT INTO bl_3nf.ce_categories (product_category_id, product_category_name,
    source_system, source_entity, product_category_name_src_id, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_categories_id'), src.category_name,
    'SA_US_SALES', 'SRC_US_SALES', src.category_src_id, CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(product_category, ''), 'n. a.') AS category_name,
            COALESCE(NULLIF(product_category, ''), 'n. a.') AS category_src_id
          FROM sa_us_sales.src_us_sales WHERE product_category IS NOT NULL) src
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_categories tgt
    WHERE tgt.product_category_name_src_id = src.category_src_id
      AND tgt.source_system = 'SA_US_SALES');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    CALL bl_cl.p_log('load_ce_categories', 'SUCCESS', v_rows_ins + v_rows_upd,
    'Inserted: ' || v_rows_ins || ', Updated: ' || v_rows_upd);
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_ce_categories', 'ERROR', NULL, SQLERRM);
END;
$$;
```

	123 product_category_id	Az product_category_name	Az source_system	Az source_entity	Az product_category_name_src_id	② insert_dt	② update_dt
1	-1	n. a.	MANUAL	MANUAL	n. a.	1900-01-01	1900-01-01
2	100	Sports	SA_INT_SALES	SRC_INT_SALES	Sports	2026-07-19	2026-07-19
3	101	Groceries	SA_INT_SALES	SRC_INT_SALES	Groceries	2026-07-19	2026-07-19
4	102	Fashion	SA_INT_SALES	SRC_INT_SALES	Fashion	2026-07-19	2026-07-19
5	103	Home & Kitchen	SA_INT_SALES	SRC_INT_SALES	Home & Kitchen	2026-07-19	2026-07-19
6	104	Electronics	SA_INT_SALES	SRC_INT_SALES	Electronics	2026-07-19	2026-07-19
7	105	Toys	SA_INT_SALES	SRC_INT_SALES	Toys	2026-07-19	2026-07-19
8	106	Books	SA_INT_SALES	SRC_INT_SALES	Books	2026-07-19	2026-07-19
9	107	Beauty	SA_INT_SALES	SRC_INT_SALES	Beauty	2026-07-19	2026-07-19
10	108	Sports	SA_US_SALES	SRC_US_SALES	Sports	2026-07-19	2026-07-19
11	109	Groceries	SA_US_SALES	SRC_US_SALES	Groceries	2026-07-19	2026-07-19
12	110	Fashion	SA_US_SALES	SRC_US_SALES	Fashion	2026-07-19	2026-07-19
13	111	Home & Kitchen	SA_US_SALES	SRC_US_SALES	Home & Kitchen	2026-07-19	2026-07-19
14	112	Electronics	SA_US_SALES	SRC_US_SALES	Electronics	2026-07-19	2026-07-19
15	113	Toys	SA_US_SALES	SRC_US_SALES	Toys	2026-07-19	2026-07-19
16	114	Books	SA_US_SALES	SRC_US_SALES	Books	2026-07-19	2026-07-19
17	115	Beauty	SA_US_SALES	SRC_US_SALES	Beauty	2026-07-19	2026-07-19

-CE_COUNTRIES

INT loads real country values; US has no country column, so a single synthetic 'US' country row is inserted to anchor US geography. Two-block structure, INT real + US placeholder.

```

CREATE OR REPLACE PROCEDURE bl_cl.load_ce_countries()
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows_ins INT := 0;
    v_rows_upd INT := 0;
    v_tmp      INT := 0;
BEGIN
    -- INT
    UPDATE bl_3nf.ce_countries tgt
    SET country_name = src.country_name, update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(country, ''), 'n. a.') AS country_name,
            COALESCE(NULLIF(country, ''), 'n. a.') AS country_src_id
          FROM sa_int_sales.src_int_sales WHERE country IS NOT NULL) src
    WHERE tgt.country_name_src_id = src.country_src_id
      AND tgt.source_system = 'SA_INT_SALES'
      AND tgt.country_name IS DISTINCT FROM src.country_name;
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;

    INSERT INTO bl_3nf.ce_countries (country_id, country_name, source_system,
                                     source_entity, country_name_src_id, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_countries_id'), src.country_name,
           'SA_INT_SALES', 'SRC_INT_SALES', src.country_src_id, CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(country, ''), 'n. a.') AS country_name,
            COALESCE(NULLIF(country, ''), 'n. a.') AS country_src_id
          FROM sa_int_sales.src_int_sales WHERE country IS NOT NULL) src
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_countries tgt
                      WHERE tgt.country_name_src_id = src.country_src_id
                        AND tgt.source_system = 'SA_INT_SALES');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    -- US - single 'US' country
    INSERT INTO bl_3nf.ce_countries (country_id, country_name, source_system,
                                     source_entity, country_name_src_id, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_countries_id'), 'US',
           'SA_US_SALES', 'SRC_US_SALES', 'US', CURRENT_DATE, CURRENT_DATE
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_countries tgt
                      WHERE tgt.country_name_src_id = 'US' AND tgt.source_system = 'SA_US_SALES');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    CALL bl_cl.p_log('load_ce_countries', 'SUCCESS', v_rows_ins + v_rows_upd,
                    'Inserted: ' || v_rows_ins || ', Updated: ' || v_rows_upd);
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_ce_countries', 'ERROR', NULL, SQLERRM);
END;
$$;
select * from bl_3nf.ce_countries

```

country_id	AZ country_name	AZ source_system	AZ source_entity	AZ country_name_src_id	insert_dt	update_dt
-1	n. a.	MANUAL	MANUAL	n. a.	1900-01-01	1900-01-01
100	Australia	SA_INT_SALES	SRC_INT_SALES	Australia	2026-07-19	2026-07-19
101	UAE	SA_INT_SALES	SRC_INT_SALES	UAE	2026-07-19	2026-07-19
102	Germany	SA_INT_SALES	SRC_INT_SALES	Germany	2026-07-19	2026-07-19
103	France	SA_INT_SALES	SRC_INT_SALES	France	2026-07-19	2026-07-19
104	Canada	SA_INT_SALES	SRC_INT_SALES	Canada	2026-07-19	2026-07-19
105	Saudi Arabia	SA_INT_SALES	SRC_INT_SALES	Saudi Arabia	2026-07-19	2026-07-19
106	India	SA_INT_SALES	SRC_INT_SALES	India	2026-07-19	2026-07-19
107	United Kingdom	SA_INT_SALES	SRC_INT_SALES	United Kingdom	2026-07-19	2026-07-19
108	Pakistan	SA_INT_SALES	SRC_INT_SALES	Pakistan	2026-07-19	2026-07-19
109	US	SA_US_SALES	SRC_US_SALES	US	2026-07-19	2026-07-19

-CE_CHANNELS

The FOR-loop. It iterates a UNION of distinct channels from both sources row-by-row, upserting each inside the loop.

```
-- CE_CHANNELS(FOR LOOP)

CREATE OR REPLACE PROCEDURE bl_cl.load_ce_channels()
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows_ins INT := 0;
    v_rows_upd INT := 0;
    v_tmp      INT := 0;
    rec        RECORD;
BEGIN
    FOR rec IN
        SELECT DISTINCT
            COALESCE(NULLIF(channel, ''), 'n. a.') AS channel_name,
            COALESCE(NULLIF(channel, ''), 'n. a.') AS channel_src_id,
            'SA_INT_SALES' AS source_system, 'SRC_INT_SALES' AS source_entity
        FROM sa_int_sales.src_int_sales WHERE channel IS NOT NULL
        UNION
        SELECT DISTINCT
            COALESCE(NULLIF(channel, ''), 'n. a.'),
            COALESCE(NULLIF(channel, ''), 'n. a.'),
            'SA_US_SALES', 'SRC_US_SALES'
        FROM sa_us_sales.src_us_sales WHERE channel IS NOT NULL
    LOOP
        UPDATE bl_3nf.ce_channels tgt
        SET channel = rec.channel_name, update_dt = CURRENT_DATE
        WHERE tgt.channel_src_id = rec.channel_src_id
            AND tgt.source_system = rec.source_system
            AND tgt.channel IS DISTINCT FROM rec.channel_name;
        GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;

        INSERT INTO bl_3nf.ce_channels (channel_id, channel, source_system,
            source_entity, channel_src_id, insert_dt, update_dt)
        SELECT nextval('bl_3nf.seq_channels_id'), rec.channel_name,
            rec.source_system, rec.source_entity, rec.channel_src_id,
            CURRENT_DATE, CURRENT_DATE
        WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_channels tgt
            WHERE tgt.channel_src_id = rec.channel_src_id
                AND tgt.source_system = rec.source_system);
        GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;
    END LOOP;

    CALL bl_cl.p_log('load_ce_channels', 'SUCCESS', v_rows_ins + v_rows_upd,
        'Inserted: ' || v_rows_ins || ', Updated: ' || v_rows_upd || ' (FOR loop)');
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_ce_channels', 'ERROR', NULL, SQLERRM);
END;
$$;
```

123 channel_id ▼	AZ channel ▼	AZ source_system ▼	AZ source_entity ▼	AZ channel_src_id ▼	insert_dt ▼	update_dt ▼
-1	n. a.	MANUAL	MANUAL	n. a.	1900-01-01	1900-01-01
100	Marketplace	SA_INT_SALES	SRC_INT_SALES	Marketplace	2026-07-19	2026-07-19
101	Marketplace	SA_US_SALES	SRC_US_SALES	Marketplace	2026-07-19	2026-07-19
102	Mobile App	SA_INT_SALES	SRC_INT_SALES	Mobile App	2026-07-19	2026-07-19
103	Mobile App	SA_US_SALES	SRC_US_SALES	Mobile App	2026-07-19	2026-07-19
104	Web	SA_INT_SALES	SRC_INT_SALES	Web	2026-07-19	2026-07-19
105	Web	SA_US_SALES	SRC_US_SALES	Web	2026-07-19	2026-07-19

-CE_EMPLOYEES

Two-source load of sales reps with multiple attributes (first name, last name, email). The update checks all three attributes so any change triggers a refresh.

```

CREATE OR REPLACE PROCEDURE bl_cl.load_ce_employees()
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows_ins INT := 0;
    v_rows_upd INT := 0;
    v_tmp      INT := 0;
BEGIN
    -- INT
    UPDATE bl_3nf.ce_employees tgt
    SET sales_rep_first_name = src.first_name,
        sales_rep_last_name  = src.last_name,
        sales_rep_email      = src.email,
        update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(sales_rep_id_int, ''), 'n. a.') AS rep_src_id,
            COALESCE(NULLIF(sales_rep_first_name_int, ''), 'n. a.') AS first_name,
            COALESCE(NULLIF(sales_rep_last_name_int, ''), 'n. a.') AS last_name,
            COALESCE(NULLIF(sales_rep_email_int, ''), 'n. a.') AS email
          FROM sa_int_sales.src_int_sales WHERE sales_rep_id_int IS NOT NULL) src
    WHERE tgt.sales_rep_src_id = src.rep_src_id
    AND tgt.source_system = 'SA_INT_SALES'
    AND (tgt.sales_rep_first_name IS DISTINCT FROM src.first_name
        OR tgt.sales_rep_last_name IS DISTINCT FROM src.last_name
        OR tgt.sales_rep_email IS DISTINCT FROM src.email);
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;
    INSERT INTO bl_3nf.ce_employees (sales_rep_id, sales_rep_first_name, sales_rep_last_name,
        sales_rep_email, source_system, source_entity, sales_rep_src_id, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_employees_id'), src.first_name, src.last_name, src.email,
        'SA_INT_SALES', 'SRC_INT_SALES', src.rep_src_id, CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(sales_rep_id_int, ''), 'n. a.') AS rep_src_id,
            COALESCE(NULLIF(sales_rep_first_name_int, ''), 'n. a.') AS first_name,
            COALESCE(NULLIF(sales_rep_last_name_int, ''), 'n. a.') AS last_name,
            COALESCE(NULLIF(sales_rep_email_int, ''), 'n. a.') AS email
          FROM sa_int_sales.src_int_sales WHERE sales_rep_id_int IS NOT NULL) src
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_employees tgt
        WHERE tgt.sales_rep_src_id = src.rep_src_id AND tgt.source_system = 'SA_INT_SALES');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;
    -- US
    UPDATE bl_3nf.ce_employees tgt
    SET sales_rep_first_name = src.first_name,
        sales_rep_last_name  = src.last_name,
        sales_rep_email      = src.email,
        update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(sales_rep_id, ''), 'n. a.') AS rep_src_id,
            COALESCE(NULLIF(sales_rep_first_name, ''), 'n. a.') AS first_name,
            COALESCE(NULLIF(sales_rep_last_name, ''), 'n. a.') AS last_name,
            COALESCE(NULLIF(sales_rep_email, ''), 'n. a.') AS email
          FROM sa_us_sales.src_us_sales WHERE sales_rep_id IS NOT NULL) src
    WHERE tgt.sales_rep_src_id = src.rep_src_id
    AND tgt.source_system = 'SA_US_SALES'
    AND (tgt.sales_rep_first_name IS DISTINCT FROM src.first_name
        OR tgt.sales_rep_last_name IS DISTINCT FROM src.last_name
        OR tgt.sales_rep_email IS DISTINCT FROM src.email);
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;
    INSERT INTO bl_3nf.ce_employees (sales_rep_id, sales_rep_first_name, sales_rep_last_name,
        sales_rep_email, source_system, source_entity, sales_rep_src_id, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_employees_id'), src.first_name, src.last_name, src.email,
        'SA_US_SALES', 'SRC_US_SALES', src.rep_src_id, CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(sales_rep_id, ''), 'n. a.') AS rep_src_id,
            COALESCE(NULLIF(sales_rep_first_name, ''), 'n. a.') AS first_name,
            COALESCE(NULLIF(sales_rep_last_name, ''), 'n. a.') AS last_name,
            COALESCE(NULLIF(sales_rep_email, ''), 'n. a.') AS email
          FROM sa_us_sales.src_us_sales WHERE sales_rep_id IS NOT NULL) src
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_employees tgt
        WHERE tgt.sales_rep_src_id = src.rep_src_id AND tgt.source_system = 'SA_US_SALES');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;
    CALL bl_cl.p_log('load_ce_employees', 'SUCCESS', v_rows_ins + v_rows_upd,
        'Inserted: ' || v_rows_ins || ', Updated: ' || v_rows_upd);
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_ce_employees', 'ERROR', NULL, SQLERRM);
END;
$$;

```

123 sales_rep	AZ sales_rep_first	AZ sales_rep_last_name	AZ sales_rep_email	AZ source_system	AZ source_entity	AZ sales_rep_src_id	insert_dt	update_dt
-1	n. a.	n. a.	n. a.	MANUAL	MANUAL	n. a.	1900-01-01	1900-01-01
100	Michael	Turner	michael.turner@novamart.com	SA_INT_SALES	SRC_INT_SALES	RINT032	2026-07-19	2026-07-19
101	Joshua	Turner	joshua.turner@novamart.com	SA_INT_SALES	SRC_INT_SALES	RINT040	2026-07-19	2026-07-19
102	Lauren	Stewart	lauren.stewart@novamart.com	SA_INT_SALES	SRC_INT_SALES	RINT081	2026-07-19	2026-07-19
103	Gregory	Johnson	gregory.johnson@novamart.com	SA_INT_SALES	SRC_INT_SALES	RINT047	2026-07-19	2026-07-19
104	Olivia	White	olivia.white@novamart.com	SA_INT_SALES	SRC_INT_SALES	RINT066	2026-07-19	2026-07-19
105	Gary	Rodriguez	gary.rodriguez@novamart.com	SA_INT_SALES	SRC_INT_SALES	RINT048	2026-07-19	2026-07-19
106	Rachel	Flores	rachel.flores@novamart.com	SA_INT_SALES	SRC_INT_SALES	RINT087	2026-07-19	2026-07-19
107	Justin	Clark	justin.clark@novamart.com	SA_INT_SALES	SRC_INT_SALES	RINT077	2026-07-19	2026-07-19
108	Jennifer	Johnson	jennifer.johnson@novamart.com	SA_INT_SALES	SRC_INT_SALES	RINT039	2026-07-19	2026-07-19
109	Linda	Clark	linda.clark@novamart.com	SA_INT_SALES	SRC_INT_SALES	RINT075	2026-07-19	2026-07-19
110	Jeffrey	Martin	jeffrey.martin@novamart.com	SA_INT_SALES	SRC_INT_SALES	RINT015	2026-07-19	2026-07-19
111	Thomas	Reyes	thomas.reyes@novamart.com	SA_INT_SALES	SRC_INT_SALES	RINT099	2026-07-19	2026-07-19
112	Linda	Reyes	linda.reyes@novamart.com	SA_INT_SALES	SRC_INT_SALES	RINT094	2026-07-19	2026-07-19
113	Isura	Robinson	isura.robinson@novamart.com	SA_INT_SALES	SRC_INT_SALES	RINT019	2026-07-19	2026-07-19

-CE_CUSTOMERS_SCD

SCD Type 2. Rather than overwriting, it expires the changed active version (is_active='N', sets end_dt) and inserts a new active version with an open end date (9999-12-31). Logs "Expired" instead of "Updated" to reflect the versioning.

```

CREATE OR REPLACE PROCEDURE bl_cl.load_ce_customers_scd()
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows_ins INT := 0;
    v_rows_exp INT := 0;
    v_tmp      INT := 0;
BEGIN
    -- INT
    UPDATE bl_3nf.ce_customers_scd tgt
    SET is_active = 'N', end_dt = CURRENT_DATE - 1, update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(customer_id_int, ''), 'n. a.') AS customer_src_id,
            COALESCE(NULLIF(customer_first_name_int, ''), 'n. a.') AS first_name,
            COALESCE(NULLIF(customer_last_name_int, ''), 'n. a.') AS last_name,
            COALESCE(NULLIF(customer_email_int, ''), 'n. a.') AS email,
            'n. a.' AS age, 'n. a.' AS gender, 'n. a.' AS segment
          FROM sa_int_sales.src_int_sales WHERE customer_id_int IS NOT NULL) src
    WHERE tgt.customer_src_id = src.customer_src_id
    AND tgt.source_system = 'SA_INT_SALES' AND tgt.is_active = 'Y'
    AND (tgt.customer_first_name IS DISTINCT FROM src.first_name
        OR tgt.customer_last_name IS DISTINCT FROM src.last_name
        OR tgt.customer_email IS DISTINCT FROM src.email
        OR tgt.customer_age IS DISTINCT FROM src.age
        OR tgt.customer_gender IS DISTINCT FROM src.gender
        OR tgt.customer_segment IS DISTINCT FROM src.segment);
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_exp := v_rows_exp + v_tmp;

    INSERT INTO bl_3nf.ce_customers_scd (customer_id, customer_first_name, customer_last_name,
    customer_email, customer_age, customer_gender, customer_segment, source_system,
    source_entity, customer_src_id, is_active, start_dt, end_dt, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_customers_id'), src.first_name, src.last_name, src.email,
    src.age, src.gender, src.segment, 'SA_INT_SALES', 'SRC_INT_SALES', src.customer_src_id,
    'Y', CURRENT_DATE, DATE '9999-12-31', CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(customer_id_int, ''), 'n. a.') AS customer_src_id,
            COALESCE(NULLIF(customer_first_name_int, ''), 'n. a.') AS first_name,
            COALESCE(NULLIF(customer_last_name_int, ''), 'n. a.') AS last_name,
            COALESCE(NULLIF(customer_email_int, ''), 'n. a.') AS email,
            'n. a.' AS age, 'n. a.' AS gender, 'n. a.' AS segment
          FROM sa_int_sales.src_int_sales WHERE customer_id_int IS NOT NULL) src
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_customers_scd tgt
    WHERE tgt.customer_src_id = src.customer_src_id
    AND tgt.source_system = 'SA_INT_SALES' AND tgt.is_active = 'Y');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    -- US
    UPDATE bl_3nf.ce_customers_scd tgt
    SET is_active = 'N', end_dt = CURRENT_DATE - 1, update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(customer_id, ''), 'n. a.') AS customer_src_id,
            COALESCE(NULLIF(customer_first_name, ''), 'n. a.') AS first_name,
            COALESCE(NULLIF(customer_last_name, ''), 'n. a.') AS last_name,
            COALESCE(NULLIF(customer_email, ''), 'n. a.') AS email,
            COALESCE(NULLIF(customer_age, ''), 'n. a.') AS age,
            COALESCE(NULLIF(customer_gender, ''), 'n. a.') AS gender,
            COALESCE(NULLIF(customer_segment, ''), 'n. a.') AS segment
          FROM sa_us_sales.src_us_sales WHERE customer_id IS NOT NULL) src
    WHERE tgt.customer_src_id = src.customer_src_id
    AND tgt.source_system = 'SA_US_SALES' AND tgt.is_active = 'Y'
    AND (tgt.customer_first_name IS DISTINCT FROM src.first_name
        OR tgt.customer_last_name IS DISTINCT FROM src.last_name
        OR tgt.customer_email IS DISTINCT FROM src.email
        OR tgt.customer_age IS DISTINCT FROM src.age
        OR tgt.customer_gender IS DISTINCT FROM src.gender
        OR tgt.customer_segment IS DISTINCT FROM src.segment);
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_exp := v_rows_exp + v_tmp;

    INSERT INTO bl_3nf.ce_customers_scd (customer_id, customer_first_name, customer_last_name,
    customer_email, customer_age, customer_gender, customer_segment, source_system,
    source_entity, customer_src_id, is_active, start_dt, end_dt, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_customers_id'), src.first_name, src.last_name, src.email,
    src.age, src.gender, src.segment, 'SA_US_SALES', 'SRC_US_SALES', src.customer_src_id,
    'Y', CURRENT_DATE, DATE '9999-12-31', CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(customer_id, ''), 'n. a.') AS customer_src_id,
            COALESCE(NULLIF(customer_first_name, ''), 'n. a.') AS first_name,
            COALESCE(NULLIF(customer_last_name, ''), 'n. a.') AS last_name,
            COALESCE(NULLIF(customer_email, ''), 'n. a.') AS email,
            COALESCE(NULLIF(customer_age, ''), 'n. a.') AS age,
            COALESCE(NULLIF(customer_gender, ''), 'n. a.') AS gender,
            COALESCE(NULLIF(customer_segment, ''), 'n. a.') AS segment
          FROM sa_us_sales.src_us_sales WHERE customer_id IS NOT NULL) src
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_customers_scd tgt
    WHERE tgt.customer_src_id = src.customer_src_id
    AND tgt.source_system = 'SA_US_SALES' AND tgt.is_active = 'Y');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    CALL bl_cl.p_log('load_ce_customers_scd', 'SUCCESS', v_rows_ins + v_rows_exp,
    'Inserted: ' || v_rows_ins || ', Expired: ' || v_rows_exp);
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_ce_customers_scd', 'ERROR', NULL, SQLERRM);
END;
$$;

```

123 custo	AZ custom	AZ cus	AZ customer_email	AZ	AZ c	AZ	AZ source_	AZ source_	AZ custc	AZ	start_dt	end_dt	insert_dt
1	n. a.	n. a.	n. a.	n. a.	n. a.	n. a.	MANUAL	MANUAL	n. a.	Y	1900-01-01 GMT+02:	9999-12-31	1900-01-01
100	Matthew	Sanchez	matthew.sanchez000001@gmail.com	n. a.	n. a.	n. a.	SA_INT_SALES	SRC_INT_SALES	CINT000001	Y	2026-07-19 GMT+03:	9999-12-31	2026-07-19
101	Diane	Green	diane.green000002@gmail.com	n. a.	n. a.	n. a.	SA_INT_SALES	SRC_INT_SALES	CINT000002	Y	2026-07-19 GMT+03:	9999-12-31	2026-07-19
102	Rachel	Nguyen	rachel.nguyen000003@yahoo.com	n. a.	n. a.	n. a.	SA_INT_SALES	SRC_INT_SALES	CINT000003	Y	2026-07-19 GMT+03:	9999-12-31	2026-07-19
103	Edward	Rivera	edward.rivera000004@yahoo.com	n. a.	n. a.	n. a.	SA_INT_SALES	SRC_INT_SALES	CINT000004	Y	2026-07-19 GMT+03:	9999-12-31	2026-07-19
104	Claire	Leroy	claire.leroy000005@hotmail.com	n. a.	n. a.	n. a.	SA_INT_SALES	SRC_INT_SALES	CINT000005	Y	2026-07-19 GMT+03:	9999-12-31	2026-07-19
105	Andrea	Krause	andrea.krause000006@gmail.com	n. a.	n. a.	n. a.	SA_INT_SALES	SRC_INT_SALES	CINT000006	Y	2026-07-19 GMT+03:	9999-12-31	2026-07-19
106	Louis	Legrand	louis.legrand000007@gmail.com	n. a.	n. a.	n. a.	SA_INT_SALES	SRC_INT_SALES	CINT000007	Y	2026-07-19 GMT+03:	9999-12-31	2026-07-19
107	Hamad	Al-Saud	hamad.al-saud000008@outlook.com	n. a.	n. a.	n. a.	SA_INT_SALES	SRC_INT_SALES	CINT000008	Y	2026-07-19 GMT+03:	9999-12-31	2026-07-19
108	Sophie	Robert	sophie.robert000010@gmail.com	n. a.	n. a.	n. a.	SA_INT_SALES	SRC_INT_SALES	CINT000010	Y	2026-07-19 GMT+03:	9999-12-31	2026-07-19
109	Nicole	Bonnet	nicole.bonnet000011@yahoo.com	n. a.	n. a.	n. a.	SA_INT_SALES	SRC_INT_SALES	CINT000011	Y	2026-07-19 GMT+03:	9999-12-31	2026-07-19
110	Emma	Garcia	emma.garcia000012@gmail.com	n. a.	n. a.	n. a.	SA_INT_SALES	SRC_INT_SALES	CINT000012	Y	2026-07-19 GMT+03:	9999-12-31	2026-07-19
111	Aurelie	Petit	aurelie.petit000013@gmail.com	n. a.	n. a.	n. a.	SA_INT_SALES	SRC_INT_SALES	CINT000013	Y	2026-07-19 GMT+03:	9999-12-31	2026-07-19
112	Hamad	Al-Qasimi	hamad.al-qasimi000015@hotmail.com	n. a.	n. a.	n. a.	SA_INT_SALES	SRC_INT_SALES	CINT000015	Y	2026-07-19 GMT+03:	9999-12-31	2026-07-19

-CE_SUBCATEGORIES

Child of categories. Joins back to ce_categories on the parent's src_id to resolve product_category_id, then upserts.

123 prod	AZ product_subcategory_name	123 product_categ	AZ source_system	AZ source_entity	AZ product_subcategory_name_src_id	insert_dt	update_dt
100	Sportswear	100	SA_INT_SALES	SRC_INT_SALES	Sportswear	2026-07-19	2026-07-19
101	Appliances	103	SA_INT_SALES	SRC_INT_SALES	Appliances	2026-07-19	2026-07-19
102	Accessories	104	SA_INT_SALES	SRC_INT_SALES	Accessories	2026-07-19	2026-07-19
103	Furniture	103	SA_INT_SALES	SRC_INT_SALES	Furniture	2026-07-19	2026-07-19
104	Fiction	106	SA_INT_SALES	SRC_INT_SALES	Fiction	2026-07-19	2026-07-19
105	Decor	103	SA_INT_SALES	SRC_INT_SALES	Decor	2026-07-19	2026-07-19
106	Mobile	104	SA_INT_SALES	SRC_INT_SALES	Mobile	2026-07-19	2026-07-19
107	Beverages	101	SA_INT_SALES	SRC_INT_SALES	Beverages	2026-07-19	2026-07-19
108	Snacks	101	SA_INT_SALES	SRC_INT_SALES	Snacks	2026-07-19	2026-07-19
109	Fitness	100	SA_INT_SALES	SRC_INT_SALES	Fitness	2026-07-19	2026-07-19
110	Equipment	100	SA_INT_SALES	SRC_INT_SALES	Equipment	2026-07-19	2026-07-19
111	Skincare	107	SA_INT_SALES	SRC_INT_SALES	Skincare	2026-07-19	2026-07-19
112	Cookware	103	SA_INT_SALES	SRC_INT_SALES	Cookware	2026-07-19	2026-07-19

```

CREATE OR REPLACE PROCEDURE bl_cl.load_ce_subcategories()
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows_ins INT := 0;
    v_rows_upd INT := 0;
    v_tmp      INT := 0;
BEGIN
    -- INT
    UPDATE bl_3nf.ce_subcategories tgt
    SET product_subcategory_name = src.subcategory_name,
        product_category_id = cat.product_category_id,
        update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(category, ''), 'n. a.') AS category_name,
            COALESCE(NULLIF(subcategory, ''), 'n. a.') AS subcategory_name,
            COALESCE(NULLIF(subcategory, ''), 'n. a.') AS subcategory_src_id
          FROM sa_int_sales.src_int_sales WHERE subcategory IS NOT NULL) src
    JOIN bl_3nf.ce_categories cat
    ON cat.product_category_name_src_id = src.category_name AND cat.source_system = 'SA_INT_SALES'
    WHERE tgt.product_subcategory_name_src_id = src.subcategory_src_id
    AND tgt.source_system = 'SA_INT_SALES'
    AND (tgt.product_subcategory_name IS DISTINCT FROM src.subcategory_name
    OR tgt.product_category_id IS DISTINCT FROM cat.product_category_id);
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;

    INSERT INTO bl_3nf.ce_subcategories (product_subcategory_id, product_subcategory_name,
        product_category_id, source_system, source_entity, product_subcategory_name_src_id,
        insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_subcategories_id'), src.subcategory_name, cat.product_category_id,
        'SA_INT_SALES', 'SRC_INT_SALES', src.subcategory_src_id, CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(category, ''), 'n. a.') AS category_name,
            COALESCE(NULLIF(subcategory, ''), 'n. a.') AS subcategory_name,
            COALESCE(NULLIF(subcategory, ''), 'n. a.') AS subcategory_src_id
          FROM sa_int_sales.src_int_sales WHERE subcategory IS NOT NULL) src
    JOIN bl_3nf.ce_categories cat
    ON cat.product_category_name_src_id = src.category_name AND cat.source_system = 'SA_INT_SALES'
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_subcategories tgt
        WHERE tgt.product_subcategory_name_src_id = src.subcategory_src_id
        AND tgt.source_system = 'SA_INT_SALES');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    -- US
    UPDATE bl_3nf.ce_subcategories tgt
    SET product_subcategory_name = src.subcategory_name,
        product_category_id = cat.product_category_id,
        update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(product_category, ''), 'n. a.') AS category_name,
            COALESCE(NULLIF(product_subcategory, ''), 'n. a.') AS subcategory_name,
            COALESCE(NULLIF(product_subcategory, ''), 'n. a.') AS subcategory_src_id
          FROM sa_us_sales.src_us_sales WHERE product_subcategory IS NOT NULL) src
    JOIN bl_3nf.ce_categories cat
    ON cat.product_category_name_src_id = src.category_name AND cat.source_system = 'SA_US_SALES'
    WHERE tgt.product_subcategory_name_src_id = src.subcategory_src_id
    AND tgt.source_system = 'SA_US_SALES'
    AND (tgt.product_subcategory_name IS DISTINCT FROM src.subcategory_name
    OR tgt.product_category_id IS DISTINCT FROM cat.product_category_id);
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;

    INSERT INTO bl_3nf.ce_subcategories (product_subcategory_id, product_subcategory_name,
        product_category_id, source_system, source_entity, product_subcategory_name_src_id,
        insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_subcategories_id'), src.subcategory_name, cat.product_category_id,
        'SA_US_SALES', 'SRC_US_SALES', src.subcategory_src_id, CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(product_category, ''), 'n. a.') AS category_name,
            COALESCE(NULLIF(product_subcategory, ''), 'n. a.') AS subcategory_name,
            COALESCE(NULLIF(product_subcategory, ''), 'n. a.') AS subcategory_src_id
          FROM sa_us_sales.src_us_sales WHERE product_subcategory IS NOT NULL) src
    JOIN bl_3nf.ce_categories cat
    ON cat.product_category_name_src_id = src.category_name AND cat.source_system = 'SA_US_SALES'
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_subcategories tgt
        WHERE tgt.product_subcategory_name_src_id = src.subcategory_src_id
        AND tgt.source_system = 'SA_US_SALES');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    CALL bl_cl.p_log('load_ce_subcategories', 'SUCCESS', v_rows_ins + v_rows_upd,
        'Inserted: ' || v_rows_ins || ', Updated: ' || v_rows_upd);
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_ce_subcategories', 'ERROR', NULL, SQLERRM);
END;
$$;

```

-CE_STATES

Child of countries. INT creates synthetic states per country (named country || '_state') so INT cities can trace back to their country despite having no state in the source. US loads real states, all linked to the synthetic 'US' country.

```
CREATE OR REPLACE PROCEDURE bl_cl.load_ce_states()
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows_ins INT := 0;
    v_rows_upd INT := 0;
    v_tmp      INT := 0;
BEGIN
    -- INT
    UPDATE bl_3nf.ce_states tgt
    SET state_name = src.state_name,
        country_id = cnt.country_id,
        update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(country, ''), 'n. a.') || '_state' AS state_name,
            COALESCE(NULLIF(country, ''), 'n. a.') || '_state' AS state_src_id,
            COALESCE(NULLIF(country, ''), 'n. a.') AS country_key
         FROM sa_int_sales.src_int_sales WHERE country IS NOT NULL) src
    JOIN bl_3nf.ce_countries cnt
    ON cnt.country_name_src_id = src.country_key
    AND cnt.source_system = 'SA_INT_SALES'
    WHERE tgt.state_name_src_id = src.state_src_id
    AND tgt.source_system = 'SA_INT_SALES'
    AND (tgt.state_name IS DISTINCT FROM src.state_name
         OR tgt.country_id IS DISTINCT FROM cnt.country_id);
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;

    INSERT INTO bl_3nf.ce_states (state_id, state_name, country_id, source_system,
        source_entity, state_name_src_id, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_states_id'), src.state_name, cnt.country_id,
        'SA_INT_SALES', 'SRC_INT_SALES', src.state_src_id, CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(country, ''), 'n. a.') || '_state' AS state_name,
            COALESCE(NULLIF(country, ''), 'n. a.') || '_state' AS state_src_id,
            COALESCE(NULLIF(country, ''), 'n. a.') AS country_key
         FROM sa_int_sales.src_int_sales WHERE country IS NOT NULL) src
    JOIN bl_3nf.ce_countries cnt
    ON cnt.country_name_src_id = src.country_key
    AND cnt.source_system = 'SA_INT_SALES'
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_states tgt
        WHERE tgt.state_name_src_id = src.state_src_id
        AND tgt.source_system = 'SA_INT_SALES');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    -- US
    UPDATE bl_3nf.ce_states tgt
    SET state_name = src.state_name,
        country_id = cnt.country_id,
        update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(state, ''), 'n. a.') AS state_name,
            COALESCE(NULLIF(state, ''), 'n. a.') AS state_src_id
         FROM sa_us_sales.src_us_sales WHERE state IS NOT NULL) src
    JOIN bl_3nf.ce_countries cnt
    ON cnt.country_name_src_id = 'US' AND cnt.source_system = 'SA_US_SALES'
    WHERE tgt.state_name_src_id = src.state_src_id
    AND tgt.source_system = 'SA_US_SALES'
    AND (tgt.state_name IS DISTINCT FROM src.state_name
         OR tgt.country_id IS DISTINCT FROM cnt.country_id);
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;

    INSERT INTO bl_3nf.ce_states (state_id, state_name, country_id, source_system,
        source_entity, state_name_src_id, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_states_id'), src.state_name, cnt.country_id,
        'SA_US_SALES', 'SRC_US_SALES', src.state_src_id, CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(state, ''), 'n. a.') AS state_name,
            COALESCE(NULLIF(state, ''), 'n. a.') AS state_src_id
         FROM sa_us_sales.src_us_sales WHERE state IS NOT NULL) src
    JOIN bl_3nf.ce_countries cnt
    ON cnt.country_name_src_id = 'US' AND cnt.source_system = 'SA_US_SALES'
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_states tgt
        WHERE tgt.state_name_src_id = src.state_src_id
        AND tgt.source_system = 'SA_US_SALES');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    CALL bl_cl.p_log('load_ce_states', 'SUCCESS', v_rows_ins + v_rows_upd,
        'Inserted: ' || v_rows_ins || ', Updated: ' || v_rows_upd);
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_ce_states', 'ERROR', NULL, SQLERRM);
END;
$;$
```

-CE_SUBCHANNELS

Child of channels. Joins to ce_channels to resolve channel_id, then upserts subchannels from both sources.

123 subchannel_id	AZ subchannel	123 channel_id	AZ source_system	AZ source_entity	AZ subchannel_src_id	insert_dt	update_dt
-1	n. a.	-1	MANUAL	MANUAL	n. a.	1900-01-01	1900-01-01
100	Mobile Web	104	SA_INT_SALES	SRC_INT_SALES	Mobile Web	2026-07-19	2026-07-19
101	iOS App	102	SA_INT_SALES	SRC_INT_SALES	iOS App	2026-07-19	2026-07-19
102	BuyEverything	100	SA_INT_SALES	SRC_INT_SALES	BuyEverything	2026-07-19	2026-07-19
103	Android App	102	SA_INT_SALES	SRC_INT_SALES	Android App	2026-07-19	2026-07-19
104	Desktop Web	104	SA_INT_SALES	SRC_INT_SALES	Desktop Web	2026-07-19	2026-07-19
105	third-party business	100	SA_INT_SALES	SRC_INT_SALES	third-party business	2026-07-19	2026-07-19
106	Mobile Web	105	SA_US_SALES	SRC_US_SALES	Mobile Web	2026-07-19	2026-07-19
107	iOS App	103	SA_US_SALES	SRC_US_SALES	iOS App	2026-07-19	2026-07-19
108	BuyEverything	101	SA_US_SALES	SRC_US_SALES	BuyEverything	2026-07-19	2026-07-19
109	Desktop Web	105	SA_US_SALES	SRC_US_SALES	Desktop Web	2026-07-19	2026-07-19
110	Android App	103	SA_US_SALES	SRC_US_SALES	Android App	2026-07-19	2026-07-19
111	third-party business	101	SA_US_SALES	SRC_US_SALES	third-party business	2026-07-19	2026-07-19

```

CREATE OR REPLACE PROCEDURE bl_cl.load_ce_subchannels()
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows_ins INT := 0;
    v_rows_upd INT := 0;
    v_tmp      INT := 0;
BEGIN
    -- INT
    UPDATE bl_3nf.ce_subchannels tgt
    SET subchannel = src.subchannel_name, channel_id = ch.channel_id, update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(channel, ''), 'n. a.') AS channel_name,
            COALESCE(NULLIF(subchannel, ''), 'n. a.') AS subchannel_name,
            COALESCE(NULLIF(subchannel, ''), 'n. a.') AS subchannel_src_id
          FROM sa_int_sales.src_int_sales WHERE subchannel IS NOT NULL) src
    JOIN bl_3nf.ce_channels ch
      ON ch.channel_src_id = src.channel_name AND ch.source_system = 'SA_INT_SALES'
    WHERE tgt.subchannel_src_id = src.subchannel_src_id
      AND tgt.source_system = 'SA_INT_SALES'
      AND (tgt.subchannel IS DISTINCT FROM src.subchannel_name
           OR tgt.channel_id IS DISTINCT FROM ch.channel_id);
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;

    INSERT INTO bl_3nf.ce_subchannels (subchannel_id, subchannel, channel_id, source_system,
                                         source_entity, subchannel_src_id, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_subchannels_id'), src.subchannel_name, ch.channel_id,
           'SA_INT_SALES', 'SRC_INT_SALES', src.subchannel_src_id, CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(channel, ''), 'n. a.') AS channel_name,
            COALESCE(NULLIF(subchannel, ''), 'n. a.') AS subchannel_name,
            COALESCE(NULLIF(subchannel, ''), 'n. a.') AS subchannel_src_id
          FROM sa_int_sales.src_int_sales WHERE subchannel IS NOT NULL) src
    JOIN bl_3nf.ce_channels ch
      ON ch.channel_src_id = src.channel_name AND ch.source_system = 'SA_INT_SALES'
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_subchannels tgt
                      WHERE tgt.subchannel_src_id = src.subchannel_src_id AND tgt.source_system = 'SA_INT_SALES');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    -- US
    UPDATE bl_3nf.ce_subchannels tgt
    SET subchannel = src.subchannel_name, channel_id = ch.channel_id, update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(channel, ''), 'n. a.') AS channel_name,
            COALESCE(NULLIF(subchannel, ''), 'n. a.') AS subchannel_name,
            COALESCE(NULLIF(subchannel, ''), 'n. a.') AS subchannel_src_id
          FROM sa_us_sales.src_us_sales WHERE subchannel IS NOT NULL) src
    JOIN bl_3nf.ce_channels ch
      ON ch.channel_src_id = src.channel_name AND ch.source_system = 'SA_US_SALES'
    WHERE tgt.subchannel_src_id = src.subchannel_src_id
      AND tgt.source_system = 'SA_US_SALES'
      AND (tgt.subchannel IS DISTINCT FROM src.subchannel_name
           OR tgt.channel_id IS DISTINCT FROM ch.channel_id);
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;

    INSERT INTO bl_3nf.ce_subchannels (subchannel_id, subchannel, channel_id, source_system,
                                         source_entity, subchannel_src_id, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_subchannels_id'), src.subchannel_name, ch.channel_id,
           'SA_US_SALES', 'SRC_US_SALES', src.subchannel_src_id, CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(channel, ''), 'n. a.') AS channel_name,
            COALESCE(NULLIF(subchannel, ''), 'n. a.') AS subchannel_name,
            COALESCE(NULLIF(subchannel, ''), 'n. a.') AS subchannel_src_id
          FROM sa_us_sales.src_us_sales WHERE subchannel IS NOT NULL) src
    JOIN bl_3nf.ce_channels ch
      ON ch.channel_src_id = src.channel_name AND ch.source_system = 'SA_US_SALES'
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_subchannels tgt
                      WHERE tgt.subchannel_src_id = src.subchannel_src_id AND tgt.source_system = 'SA_US_SALES');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    CALL bl_cl.p_log('load_ce_subchannels', 'SUCCESS', v_rows_ins + v_rows_upd,
                    'Inserted: ' || v_rows_ins || ', Updated: ' || v_rows_upd);
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_ce_subchannels', 'ERROR', NULL, SQLERRM);
END;
$$;

```

-CE_PRODUCTS

Child of subcategories. Joins to ce_subcategories to resolve product_subcategory_id, then upserts products from both sources.

```
CREATE OR REPLACE PROCEDURE bl_cl.load_ce_products()
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows_ins INT := 0;
    v_rows_upd INT := 0;
    v_tmp      INT := 0;
BEGIN
    -- INT
    UPDATE bl_3nf.ce_products tgt
    SET product_subcategory_id = sub.product_subcategory_id, update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(product_id_int, ''), 'n. a.') AS product_src_id,
            COALESCE(NULLIF(subcategory, ''), 'n. a.')   AS subcategory_name
          FROM sa_int_sales.src_int_sales WHERE product_id_int IS NOT NULL) src
    JOIN bl_3nf.ce_subcategories sub
    ON sub.product_subcategory_name_src_id = src.subcategory_name AND sub.source_system = 'SA_INT_SALES'
    WHERE tgt.product_src_id = src.product_src_id
    AND tgt.source_system = 'SA_INT_SALES'
    AND tgt.product_subcategory_id IS DISTINCT FROM sub.product_subcategory_id;
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;

    INSERT INTO bl_3nf.ce_products (product_id, product_subcategory_id, source_system,
    source_entity, product_src_id, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_products_id'), sub.product_subcategory_id,
    'SA_INT_SALES', 'SRC_INT_SALES', src.product_src_id, CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(product_id_int, ''), 'n. a.') AS product_src_id,
            COALESCE(NULLIF(subcategory, ''), 'n. a.')   AS subcategory_name
          FROM sa_int_sales.src_int_sales WHERE product_id_int IS NOT NULL) src
    JOIN bl_3nf.ce_subcategories sub
    ON sub.product_subcategory_name_src_id = src.subcategory_name AND sub.source_system = 'SA_INT_SALES'
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_products tgt
    WHERE tgt.product_src_id = src.product_src_id AND tgt.source_system = 'SA_INT_SALES');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    -- US
    UPDATE bl_3nf.ce_products tgt
    SET product_subcategory_id = sub.product_subcategory_id, update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(product_id, ''), 'n. a.')      AS product_src_id,
            COALESCE(NULLIF(product_subcategory, ''), 'n. a.') AS subcategory_name
          FROM sa_us_sales.src_us_sales WHERE product_id IS NOT NULL) src
    JOIN bl_3nf.ce_subcategories sub
    ON sub.product_subcategory_name_src_id = src.subcategory_name AND sub.source_system = 'SA_US_SALES'
    WHERE tgt.product_src_id = src.product_src_id
    AND tgt.source_system = 'SA_US_SALES'
    AND tgt.product_subcategory_id IS DISTINCT FROM sub.product_subcategory_id;
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;

    INSERT INTO bl_3nf.ce_products (product_id, product_subcategory_id, source_system,
    source_entity, product_src_id, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_products_id'), sub.product_subcategory_id,
    'SA_US_SALES', 'SRC_US_SALES', src.product_src_id, CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(product_id, ''), 'n. a.')      AS product_src_id,
            COALESCE(NULLIF(product_subcategory, ''), 'n. a.') AS subcategory_name
          FROM sa_us_sales.src_us_sales WHERE product_id IS NOT NULL) src
    JOIN bl_3nf.ce_subcategories sub
    ON sub.product_subcategory_name_src_id = src.subcategory_name AND sub.source_system = 'SA_US_SALES'
    WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_products tgt
    WHERE tgt.product_src_id = src.product_src_id AND tgt.source_system = 'SA_US_SALES');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    CALL bl_cl.p_log('load_ce_products', 'SUCCESS', v_rows_ins + v_rows_upd,
    'Inserted: ' || v_rows_ins || ', Updated: ' || v_rows_upd);
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_ce_products', 'ERROR', NULL, SQLERRM);
END;
$;$
```

123 product_id	123 product_subcategory_id	AZ source_system	AZ source_entity	AZ product_src_id	insert_dt	update_dt
-1	-1	MANUAL	MANUAL	n. a.	1900-01-01	1900-01-01
100	101	SA_INT_SALES	SRC_INT_SALES	PROD01531	2026-07-19	2026-07-19
101	117	SA_INT_SALES	SRC_INT_SALES	PROD01288	2026-07-19	2026-07-19
102	108	SA_INT_SALES	SRC_INT_SALES	PROD00870	2026-07-19	2026-07-19
103	119	SA_INT_SALES	SRC_INT_SALES	PROD02266	2026-07-19	2026-07-19
104	102	SA_INT_SALES	SRC_INT_SALES	PROD01009	2026-07-19	2026-07-19
105	111	SA_INT_SALES	SRC_INT_SALES	PROD01874	2026-07-19	2026-07-19
106	113	SA_INT_SALES	SRC_INT_SALES	PROD00939	2026-07-19	2026-07-19
107	107	SA_INT_SALES	SRC_INT_SALES	PROD00784	2026-07-19	2026-07-19
108	117	SA_INT_SALES	SRC_INT_SALES	PROD01244	2026-07-19	2026-07-19
109	107	SA_INT_SALES	SRC_INT_SALES	PROD00692	2026-07-19	2026-07-19
110	126	SA_INT_SALES	SRC_INT_SALES	PROD00500	2026-07-19	2026-07-19
111	128	SA_INT_SALES	SRC_INT_SALES	PROD00239	2026-07-19	2026-07-19
112	128	SA_INT_SALES	SRC_INT_SALES	PROD00423	2026-07-19	2026-07-19
113	126	SA_INT_SALES	SRC_INT_SALES	PROD00440	2026-07-19	2026-07-19

-CE_CITIES

Child of states, with asymmetric handling: INT cities link to their country's synthetic _state; US cities link to real states and match on state_id to rule out from same-named cities across different states (e.g. Charleston in SC vs WV). That was a problem arised while creating insert/update procedure

123 city_id	AZ city_name	123 state_id	AZ source_system	AZ source_entity	AZ city_name_src_id	insert_dt	update_dt
-1	n. a.	-1	MANUAL	MANUAL	n. a.	1900-01-01	1900-01-01
788	Hyderabad	239	SA_INT_SALES	SRC_INT_SALES	Hyderabad	2026-07-19	2026-07-19
789	Calgary	242	SA_INT_SALES	SRC_INT_SALES	Calgary	2026-07-19	2026-07-19
790	Berlin	241	SA_INT_SALES	SRC_INT_SALES	Berlin	2026-07-19	2026-07-19
791	Melbourne	244	SA_INT_SALES	SRC_INT_SALES	Melbourne	2026-07-19	2026-07-19
792	Cologne	241	SA_INT_SALES	SRC_INT_SALES	Cologne	2026-07-19	2026-07-19
793	Perth	244	SA_INT_SALES	SRC_INT_SALES	Perth	2026-07-19	2026-07-19
794	Vancouver	242	SA_INT_SALES	SRC_INT_SALES	Vancouver	2026-07-19	2026-07-19
795	Abu Dhabi	240	SA_INT_SALES	SRC_INT_SALES	Abu Dhabi	2026-07-19	2026-07-19
796	Lyon	236	SA_INT_SALES	SRC_INT_SALES	Lyon	2026-07-19	2026-07-19
797	Riyadh	238	SA_INT_SALES	SRC_INT_SALES	Riyadh	2026-07-19	2026-07-19
798	Frankfurt	241	SA_INT_SALES	SRC_INT_SALES	Frankfurt	2026-07-19	2026-07-19
799	Karachi	243	SA_INT_SALES	SRC_INT_SALES	Karachi	2026-07-19	2026-07-19
800	Sharjah	240	SA_INT_SALES	SRC_INT_SALES	Sharjah	2026-07-19	2026-07-19
801	Lahore	242	SA_INT_SALES	SRC_INT_SALES	Lahore	2026-07-19	2026-07-19

```

CREATE OR REPLACE PROCEDURE bl_cl.load_ce_cities()
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows_ins INT := 0;
    v_rows_upd INT := 0;
    v_tmp      INT := 0;
BEGIN
    -- INT
    UPDATE bl_3nf.ce_cities tgt
    SET city_name = src.city_name,
        state_id = st.state_id,
        update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(city_int, ''), 'n. a.') AS city_name,
            COALESCE(NULLIF(city_int, ''), 'n. a.') AS city_src_id,
            COALESCE(NULLIF(country, ''), 'n. a.') || '_state' AS state_key
          FROM sa_int_sales.src_int_sales WHERE city_int IS NOT NULL) src
    JOIN bl_3nf.ce_states st
      ON st.state_name_src_id = src.state_key
     AND st.source_system = 'SA_INT_SALES'
   WHERE tgt.city_name_src_id = src.city_src_id
     AND tgt.source_system = 'SA_INT_SALES'
     AND (tgt.city_name IS DISTINCT FROM src.city_name
          OR tgt.state_id IS DISTINCT FROM st.state_id);
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;

    INSERT INTO bl_3nf.ce_cities (city_id, city_name, state_id, source_system,
        source_entity, city_name_src_id, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_cities_id'), src.city_name, st.state_id,
        'SA_INT_SALES', 'SRC_INT_SALES', src.city_src_id, CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(city_int, ''), 'n. a.') AS city_name,
            COALESCE(NULLIF(city_int, ''), 'n. a.') AS city_src_id,
            COALESCE(NULLIF(country, ''), 'n. a.') || '_state' AS state_key
          FROM sa_int_sales.src_int_sales WHERE city_int IS NOT NULL) src
    JOIN bl_3nf.ce_states st
      ON st.state_name_src_id = src.state_key
     AND st.source_system = 'SA_INT_SALES'
   WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_cities tgt
                     WHERE tgt.city_name_src_id = src.city_src_id
                       AND tgt.source_system = 'SA_INT_SALES');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    -- US
    UPDATE bl_3nf.ce_cities tgt
    SET city_name = src.city_name,
        update_dt = CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(state, ''), 'n. a.') AS state_name,
            COALESCE(NULLIF(city, ''), 'n. a.') AS city_name,
            COALESCE(NULLIF(city, ''), 'n. a.') AS city_src_id
          FROM sa_us_sales.src_us_sales WHERE city IS NOT NULL) src
    JOIN bl_3nf.ce_states st
      ON st.state_name_src_id = src.state_name
     AND st.source_system = 'SA_US_SALES'
   WHERE tgt.city_name_src_id = src.city_src_id
     AND tgt.source_system = 'SA_US_SALES'
     AND tgt.state_id = st.state_id
     AND (tgt.city_name IS DISTINCT FROM src.city_name);
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd := v_rows_upd + v_tmp;

    INSERT INTO bl_3nf.ce_cities (city_id, city_name, state_id, source_system,
        source_entity, city_name_src_id, insert_dt, update_dt)
    SELECT nextval('bl_3nf.seq_cities_id'), src.city_name, st.state_id,
        'SA_US_SALES', 'SRC_US_SALES', src.city_src_id, CURRENT_DATE, CURRENT_DATE
    FROM (SELECT DISTINCT
            COALESCE(NULLIF(state, ''), 'n. a.') AS state_name,
            COALESCE(NULLIF(city, ''), 'n. a.') AS city_name,
            COALESCE(NULLIF(city, ''), 'n. a.') AS city_src_id
          FROM sa_us_sales.src_us_sales WHERE city IS NOT NULL) src
    JOIN bl_3nf.ce_states st
      ON st.state_name_src_id = src.state_name
     AND st.source_system = 'SA_US_SALES'
   WHERE NOT EXISTS (SELECT 1 FROM bl_3nf.ce_cities tgt
                     WHERE tgt.city_name_src_id = src.city_src_id
                       AND tgt.source_system = 'SA_US_SALES'
                       AND tgt.state_id = st.state_id);
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins := v_rows_ins + v_tmp;

    CALL bl_cl.p_log('load_ce_cities', 'SUCCESS', v_rows_ins + v_rows_upd,
        'Inserted: ' || v_rows_ins || ', Updated: ' || v_rows_upd);
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_ce_cities', 'ERROR', NULL, SQLERRM);
END;
$$;

```